

Dossier d'Organisation et de Planification : Projet ZeroStrike

Développement d'une application de jeu multijoueur distribuée

UE : Management de Projet Informatique

BUT 2 Informatique - Parcours B

Présenté par : Ilian EL BOUAZZAOUI PRIEUR, Ibrahim MAHAMAT

Groupe : TP4E-G1

Date de remise : 14 Mars 2026

Sommaire

1. Introduction	1
3. Analyse des Dépendances (PERT)	3
4. Calendrier de Réalisation (GANTT)	5
5. Répartition des Rôles	6
6. Organisation Technique et Outils	6
7. Gestion des Risques	7
8. Conclusion	7

1. Introduction

Le projet **ZeroStrike** s'inscrit dans le cadre du développement d'une application distribuée interactive. Il s'agit d'un jeu multijoueur en temps réel fonctionnant sur une architecture asymétrique : un écran principal (Display) diffuse l'environnement de jeu, tandis que les joueurs utilisent leurs smartphones comme contrôleurs (Mobile) en rejoignant des salons virtuels.

La réussite d'un tel système, mêlant développement Fullstack, conteneurisation et communication réseau, nécessite une méthodologie de travail rigoureuse. L'objectif de ce document est d'explicitier notre organisation interne : de la répartition des tâches (WBS) à la

gestion des dépendances (PERT) et du temps (GANTT), afin de garantir une livraison fonctionnelle et documentée pour le 14 mars 2026.

2. Planification Détaillée (WBS)

Pour assurer une progression constante, le projet a été découpé en grandes phases techniques et non-techniques, elles-mêmes divisées en sous-tâches.

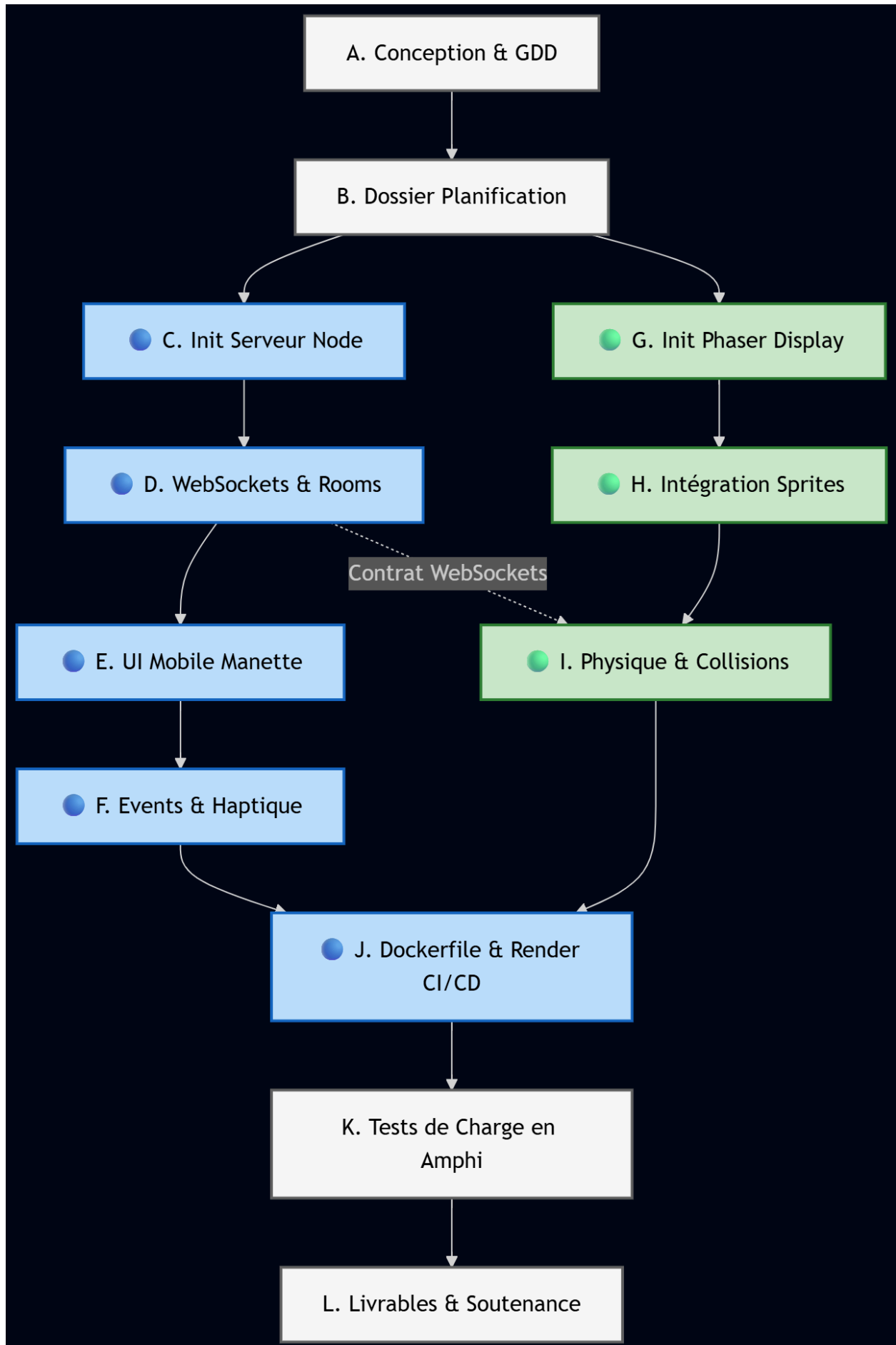
- **Phase de Conception (Non-technique)** : Définition des règles métier, modélisation de l'architecture MVC, et réalisation des maquettes UI/UX pour l'expérience multijoueur en amphi.
 - *Lien vers nos maquettes d'interface :*
https://excalidraw.com/#json=s23slHJ3JYVA4S9FA5YHg.13OauevFaWFgqgnmc_vcow
- **Phase d'Infrastructure & Réseau** : Mise en place du socle serveur sous Node.js avec le framework Express. L'effort principal réside dans l'implémentation de Socket.io pour la gestion des événements bidirectionnels et du système de "Rooms" (salons privés par code ou QR Code).
- **Phase de Développement Frontend** : Implémentation du client-display utilisant le moteur graphique Phaser pour le rendu des sprites animés, et développement du client-mobile avec Vite pour une interface tactile réactive.
- **Phase DevOps** : Industrialisation de l'application. Création d'un Dockerfile optimisé (basé sur Node Alpine) et mise en place d'un déploiement continu (CI/CD) sur la plateforme Render.
- **Phase de Finalisation** : Recette, tests de charge multijoueur en conditions réelles et rédaction de la documentation technique.

Tableau récapitulatif des tâches et estimations :

Code	Nom de la tâche	Description	Est. (h)	Responsable(s)
A	Conception et Maquettage	Diagrammes UML, interfaces Excalidraw	15h	Équipe complète
B	Socle Serveur Backend	Initialisation Node.js, Express, Routage	10h	Iljan
C	Logique Réseau (WebSockets)	Connexions Socket.io, système de Rooms	20h	Iljan

D	Frontend : Display	Moteur Phaser, gestion sprites 16-bit	25h	Ibrahim
E	Frontend : Mobile	UI manette, envoi d'événements tactiles	15h	Ilian
F	DevOps : Docker & Déploiement	Création image Docker, config Render	10h	Ilian
G	Tests et Recette	Débogage réseau, vérification de version	15h	Équipe complète
H	Rapport et Soutenance	Rédaction du livrable final	10h	Équipe complète

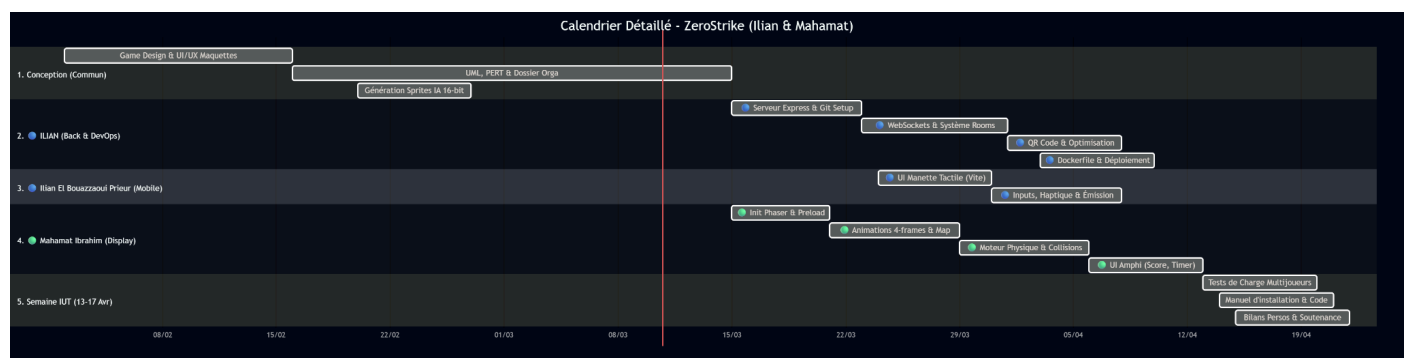
3. Analyse des Dépendances (PERT)



L'élaboration de notre diagramme de PERT a permis de mettre en évidence le **chemin critique** et les interdépendances fortes de notre application distribuée. L'architecture de **ZeroStrike** repose sur un couplage étroit entre le serveur et les clients via le protocole WebSocket.

De fait, l'infrastructure réseau (Tâche D) constitue le pivot central : il est techniquement impossible de finaliser la physique du moteur de jeu de **Ibrahim** (Tâche I) ou la réactivité des contrôleurs tactiles d'**Ilian** (Tâche F) sans qu'un contrat d'interface (API Events) ne soit parfaitement opérationnel. Le socle Backend (Tâche C) centralise l'intégralité du routage et de la gestion des salons (Rooms), créant un goulot d'étranglement initial que nous avons traité en priorité absolue pour débloquer le développement parallèle.

4. Calendrier de Réalisation (GANTT)



Le diagramme de GANTT traduit notre logique de dépendances en un rétroplanning opérationnel, aligné sur les jalons imposés par le calendrier pédagogique (rendu du dossier le 14 mars et soutenance finale mi-avril). Cette planification démontre notre stratégie d'optimisation des ressources : malgré une équipe réduite à deux membres, la **parallélisation des tâches** nous permet de maintenir une vélocité élevée.

Dès la validation du socle serveur et du contrat d'interface WebSocket (structure des paquets JSON et nomenclature des événements), le développement s'articule en deux flux asynchrones :

- **Le flux "Infrastructure & Contrôle" (Ilian)**, focalisé sur la stabilité du serveur Node.js et l'ergonomie responsive de la manette mobile.
- **Le flux "Moteur & Gameplay" (Mahamat)**, dédié à l'intégration des assets graphiques et à la logique de jeu sous Phaser.

Cette répartition par spécialité, visible par le code couleur du diagramme, minimise les conflits de fusion (Git conflicts) et réduit considérablement la durée globale du cycle de développement. Enfin, la phase de **DevOps** et de **Tests de charge** a été volontairement placée en fin de projet pour garantir que l'application ZeroStrike supporte la charge simultanée de dizaines de joueurs lors de la démonstration en amphi.

5. Répartition des Rôles

La répartition des tâches s'est appuyée sur les compétences techniques spécifiques de chaque membre de l'équipe :

- **Ilian El Bouazzaoui Prieur (Lead Backend & DevOps)** : En charge de l'architecture serveur centralisée (Node.js/Express). Responsable de la sécurité des communications réseau (Socket.io) et de l'administration système, incluant la création du Dockerfile multi-environnements et le déploiement sur le PaaS Render.
- **Mahamat Ibrahim (Lead Frontend & Game Logic)** : Responsable du client-display. Intégration du moteur graphique Phaser, gestion de la boucle de rendu en temps réel, implémentation de la physique du jeu et animation des feuilles de sprites (sprite sheets).

6. Organisation Technique et Outils

Pour soutenir cette architecture distribuée, nous avons mis en place un écosystème d'outils professionnels garantissant une communication fluide et une traçabilité du code.

Outils de communication :

- **Discord** : Outil principal pour les réunions de synchronisation (Stand-ups) et le partage d'écran lors des sessions de débogage complexe (Pair Programming réseau).
- **WhatsApp** : Utilisé pour la coordination logistique rapide et les alertes de déploiement (ex: "Le serveur de production a été mis à jour").

Versioning et Gestion de Code :

- **GitHub** : Hébergement de notre dépôt Git. Nous avons appliqué une stratégie stricte de *Feature Branching*. Le code n'est jamais poussé directement sur la branche main. Chaque nouvelle fonctionnalité (ex: feature/docker-build, feature/player-movement) possède sa propre branche.
- **Pull Requests (PR)** : Le code est fusionné via des PR exigeant la relecture d'au moins un autre membre de l'équipe, limitant ainsi le risque d'introduire des régressions dans le moteur de jeu.

-

Gestion de projet :

- **Trello / Notion** : Utilisation d'un tableau Kanban pour suivre l'avancement. Le flux de travail s'articule autour de colonnes claires (*À faire, En cours, En test, Terminé*), nous permettant de visualiser instantanément les points de blocage.

7. Gestion des Risques

L'anticipation des risques a été au cœur de notre gestion de projet :

1. **Risque Technique (Décalage de versions et Build)** : L'utilisation de technologies récentes (Vite v7) a généré des conflits avec des environnements Node.js anciens lors du déploiement.
 - *Solution* : Standardisation de l'environnement d'exécution en figeant la version avec l'image Docker node:20-alpine, garantissant la compatibilité des fonctions cryptographiques requises par Vite en production.
2. **Risque de Latence Réseau (Socket.io)** : Un volume trop important d'événements de mouvement envoyés par les mobiles risque de surcharger le serveur Render et de causer des lags sur le grand écran.
 - *Solution* : Optimisation des paquets de données et découplage de la boucle de rendu Phaser par rapport aux "ticks" réseau (implémentation d'une logique d'interpolation).
3. **Risque Temporel (Ressources Graphiques)** : Le développement des sprites en pixel art 16-bit peut prendre du retard et bloquer l'intégration Phaser.
 - *Solution* : Utilisation de "placeholders" (formes géométriques de base) pour coder la logique de déplacement sans attendre les assets graphiques finaux.

8. Conclusion

La planification détaillée du projet ZeroStrike nous a permis de surmonter les défis inhérents au développement d'une application distribuée communicante. L'utilisation d'outils de versioning robustes et d'une intégration continue conteneurisée sécurise notre cycle de développement. L'équipe est aujourd'hui pleinement confiante quant au respect de la date de livraison fixée au 14 mars 2026, avec un livrable fonctionnel répondant aux attentes du cahier des charges.